NanoAOD photon trigger objects: the unseeded-leg filter bits of HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90 can never be set

Issue report prepared for Santanu Mahata, 4 November 2026. Evidence taken from the cmssw fork checkout at commit `f99356577` (master, 2025-05-06), the upstream tag `CMSSW_13_3_1_patch1` (NanoAOD v13 production release), the upstream branches `CMSSW_10_6_X`, `CMSSW_12_4_X`, `CMSSW_12_6_X`, `CMSSW_13_0_X`, `CMSSW_14_0_X`, `CMSSW_15_0_X`, and upstream master at `ed35654b1` (2026-11-04). Fix: commit `1699a39f7` on the fork branch `claude/fervent-lamport-3oiuh6`, and the same change as `aebf3eb64` on branch `nano-photon-trigobj-unseeded` on top of upstream master. Nothing has been pushed.

Observation. In NanoAOD v13 the photon trigger objects (`TrigObj_id == 22`) document a bit for each last unseeded filter of the diphoton path (bit 14, `hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter`; bit 15, `hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter`), but no trigger object ever carries them. The sub-leading diphoton leg cannot be trigger-matched from NanoAOD.

Cause. The photon selection of the trigger-object table accepts only objects whose HLT collection is `hltEgammaCandidates`. The unseeded leg of the path is built on `hltEgammaCandidatesUnseeded`. The collection test is an exact label comparison and it is applied before the filter bits are evaluated, so every object that carries the unseeded filter labels is discarded. The filter names themselves are correct and present in all relevant HLT menus.

Scope. Three of the photon bits are unreachable in every NanoAOD version since the bits were introduced (v12, v13, v14, v15, upstream master today): 3 of 18 bits in v12 to v14, 3 of 21 in v15 and master. They belong to the unseeded legs of `HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90` and `_Mass95` in every menu from 2018 to 2025, and of `HLT_Diphoton30_18_R9IdL_AND_HE_AND_IsoCaloId` and `_Mass55` in the 2018 and 2022 menus. The identical defect for electrons was fixed in June 2023 (PR 41827); photons were missed.

Fix. One line in `PhysicsTools/NanoAOD/python/triggerObjects_cff.py`: accept both collections in the photon selection, exactly as the electron selection already does. The pull-request branch is prepared and verified to apply cleanly on upstream master.

Contents

1. How NanoAOD v13 is produced
2. Anatomy of the HLT path
3. The NanoAOD trigger-object configuration
4. Why there is never a match: the full chain
5. How many triggers and bits are affected
6. Explanations that were ruled out
7. History and affected NanoAOD versions
8. The fix and its expected effect
9. Validation plan
10. Pull request: what is ready
11. Appendix: references, line numbers, sources

1. How NanoAOD v13 is produced

NanoAOD v13 is the MiniAOD-to-NanoAOD pass of the `CMSSW_13_3_X` cycle. The production release is `CMSSW_13_3_1_patch1` (tagged 12 December 2023); its NanoAOD code is identical to the current tip of the `CMSSW_13_3_X` branch. The MC input is 13_0_X MiniAODv4; the 2022 data were re-processed from the 22Sep2023 MiniAOD under the processing string `16Dec2023`. No 2023 data were produced at v13.

```
cmsDriver.py --step NANO --mc --scenario pp \
  --eventcontent NANOAODSIM --datatier NANOAODSIM \
  --conditions 133X_mcRun3_2023_realistic_postBPix_ForNanov13_v2 --era Run3_2023 \
  --customise Configuration/DataProcessing/Utils.addMonitoring \
  --filein file:miniaod.root --fileout file:nano.root -n -1 --no_exec
```

Campaign	Conditions	Era
Run3Summer22NanoAODv13	`133X_mcRun3_2022_realistic_ForNanov13_v1`	Run3
Run3Summer22EENanoAODv13	`133X_mcRun3_2022_realistic_postEE_ForNanov13_v1`	Run3

Run3Summer23NanoAODv13	`133X_mcRun3_2023_realistic_ForNanov13_v1`	Run3_2023
Run3Summer23BPixNanoAODv13	`133X_mcRun3_2023_realistic_postBPix_ForNanov13_v2`	Run3_2023
2022 data, Run2022C to G, 16Dec2023	`133X_dataRun3 tag (exact version not confirmed)`	Run3

The NANO step is assembled by `prepare_NANO` in `Configuration/Applications/python/ConfigBuilder.py`. It schedules `nanoSequenceMC` (MC) or `nanoSequence` (data) from `PhysicsTools/NanoAOD/python/nano_cff.py` and applies `nanoAOD_customizeCommon`. The trigger objects are produced inside `triggerObjectTablesTask` by the module `triggerObjectTable` (plugin `TriggerObjectTableProducer`, `PhysicsTools/NanoAOD/plugins/TriggerObjectTableProducer.cc`), configured in `PhysicsTools/NanoAOD/python/triggerObjects_cff.py`. Its input is `unpackedPatTrigger`, the unpacked copy of `slimmedPatTrigger` from MiniAOD, a collection of `pat::TriggerObjectStandAlone`. Each stored object remembers the HLT collection it was made from (for example `hltEgammaCandidates::HLT`) and the list of HLT filter labels it passed.

For each object type (Electron, Photon, Muon, Tau, Jet, ...) the configuration gives a selection string `sel` and an ordered list `qualityBits`. Bit *i* of `TrigObj_filterBits` is set when the *i*-th entry's expression is true, typically `filter('name')`. With `skipObjectsNotPassingQualityBits = True` an object is written only if at least one bit is set.

2. Anatomy of the HLT path

In the `CMSSW_13_3_1_patch1` menu (`HLTrigger/Configuration/python/HLT_FULL_cff.py`, line 119468) the path reads:

```
HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90_v19 = cms.Path(
    HLTBeginSequence + hltL1sSingleAndDoubleEGor + hltPreDiphoton3022R9IdORIsoCaloIdANDHER9IdMass90
  + HLTDiphoton30L22R9Id85b90eORIso60CaloId15b35eANDHE12R9Id50b80eSequence
  + hltDiEG22R9Id85b90eORIso60CaloId15b35eANDHE12R9Id50b80eMass90CombMassLastFilter + HLTEndSequence )
```

The sequence (line 118801) has two legs. The leading leg (E_T > 30 GeV) is built on the L1-seeded candidates `hltEgammaCandidates`. The sub-leading leg (E_T > 22 GeV) is built on `hltEgammaCandidatesUnseeded`, which `hltDiEG22EtEta2p55UnseededFilter` (type `HLT1Photon`, `triggerType = 92`) reads directly through `inputTag = "hltEgammaCandidatesUnseeded"`. Each leg ends in two alternative last filters, one for the R9 identification branch and one for the isolation-plus-calo-ID branch, combined by OR; the two legs are then combined by the mass filter. All filters below have `saveTags = True`, so their objects are stored in the trigger summary and reach MiniAOD.

Order	Filter module	Plugin type	Candidate collection	NanoAOD photon bit (v13)
Leading leg, seeded				
1	`hltEGL1SingleAndDoubleEGOrFilter`	HLTEgammaL1TMatchFilterRegional	hltEgammaCandidates	none
2	`hltEG30L1SingleAndDoubleEGOrEtFilter`	HLTEgammaEtFilter	hltEgammaCandidates	none
3	`hltEG30LR9Id50b80eR9IdFilter`	HLTEgammaGenericFilter	hltEgammaCandidates	none
4	`hltEG30LHE12R9Id50b80eHEFilter`	HLTEgammaGenericQuadraticFilter	hltEgammaCandidates	none
5	`hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter`	HLTEgammaGenericFilter	hltEgammaCandidates	**12**
6	`hltEG30LCaloId15b35eHE12R9Id50b80eClusterShapeFilter`	HLTEgammaGenericFilter	hltEgammaCandidates	none
7	`hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter`	HLTEgammaGenericQuadraticFilter	hltEgammaCandidates	**13**
8	`hltEG30LRId85ORIso60CaloId15b35eANDHE12R9Id50b80eLegCombLastFilter`	HLTEgammaDoubleLegCombFilter	hltEgammaCandidates	none
Sub-leading leg, unseeded				
9	`hltDiEG22EtEta2p55UnseededFilter`	HLT1Photon	hltEgammaCandidatesUnseeded	none
10	`hltDiEG22R9Id50b80eR9IdUnseededFilter`	HLTEgammaGenericFilter	hltEgammaCandidatesUnseeded	none
11	`hltDiEG22HE12R9Id50b80eHEUnseededFilter`	HLTEgammaGenericQuadraticFilter	hltEgammaCandidatesUnseeded	none
12	`hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter`	HLTEgammaGenericFilter	hltEgammaCandidatesUnseeded	**15, never set**

13	hltEG22CaloId15b35eHE12R9Id50b80eClusterShapeUnseededFilter	HLTEgammaGenericFilter	hltEgammaCandidatesUnseeded	none
14	hltEG22Iso60CaloId15b35eHE12R9Id50b80eEcalIsoUnseededFilter	HLTEgammaGenericQuadraticFilter	hltEgammaCandidatesUnseeded	none
15	hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter	HLTEgammaGenericQuadraticFilter	hltEgammaCandidatesUnseeded	**14, never set**
Pair				
16	hltDiEG22R9Id85b90eORIso60CaloId15b35eANDHE12R9Id50b80eMass90CombMassLastFilter	HLTEgammaCombMassFilter	both	none

The two unseeded last filters are defined as follows (master menu `HLT_GRun_cff.py`, lines 47281 and 47362; the v13 menu is identical apart from line numbers). The candidate collection is explicit in `l1EGCand`, and the `candTag` chain leads back to `hltDiEG22EtEta2p55UnseededFilter`.

```
hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter = cms.EDFilter( "HLTEgammaGenericFilter",
    saveTags = cms.bool( True ),
    candTag  = cms.InputTag( "hltDiEG22HE12R9Id50b80eHEUnseededFilter" ),
    varTag   = cms.InputTag( 'hltEgammaR9IDUnseeded','r95x5' ),
    ...
    l1EGCand = cms.InputTag( "hltEgammaCandidatesUnseeded" )
)
hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter = cms.EDFilter( "HLTEgammaGenericQuadraticFilter",
    saveTags = cms.bool( True ),
    candTag  = cms.InputTag( "hltEG22Iso60CaloId15b35eHE12R9Id50b80eEcalIsoUnseededFilter" ),
    varTag   = cms.InputTag( "hltEgammaHollowTrackIsoUnseeded" ),
    ...
    l1EGCand = cms.InputTag( "hltEgammaCandidatesUnseeded" )
)
```

The sibling path `HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass95` shares the same sequence and differs only in the final mass filter, so it is affected in the same way. In the master menu the paths are at version `_v24` (line 86983) with unchanged filter names.

3. The NanoAOD trigger-object configuration

Photon block of `PhysicsTools/NanoAOD/python/triggerObjects_cff.py` in `CMSSW_13_3_1_patch1` (lines 81 to 106). The bit index is the 0-based position in the list.

```
Photon = cms.PSet(
    id = cms.int32(22),
    sel = cms.string("type(92) && pt > 15 && coll('hltEgammaCandidates')"),            # line 83
    l1seed = cms.string("type(-98)"),  l1deltaR = cms.double(0.3),
    skipObjectsNotPassingQualityBits = cms.bool(True),
    qualityBits = cms.VPSet(
        mksel("filter('hltEG33L1EG26HEFilter')", ...),                                    # bit 0
        ...                                                                               # bits 1-11
        mksel("filter('hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter')", ...),           # bit 12, seeded
        mksel("filter('hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter')", ...),  # bit 13, seeded
        mksel("filter('hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter')", ...), # bit 14, UNSEEDED (line 102)
        mksel("filter('hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter')", ...),      # bit 15, UNSEEDED (line 103)
        mksel("filter('hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter')", ...),   # bit 16, seeded, 2022 menu names
        mksel("filter('hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter')", ...) # bit 17, UNSEEDED (line 105)
    )
),
```

For comparison, the Electron block (line 57 in v13, line 61 in upstream master) already accepts both collections, because its last bit (the unseeded leg of `HLT_DoubleEle*_CaloIdL_MW`) needs it:

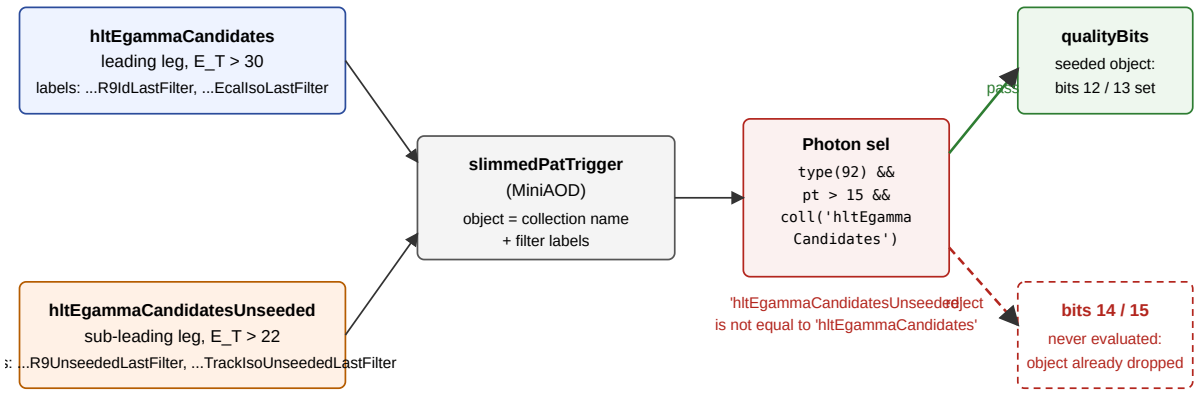
```
sel = cms.string("type(92) && pt > 7 && (coll('hltEgammaCandidates') || coll('hltEgammaCandidatesUnseeded')) && (filter('*PixelMatchFilter') || filter('*PixelMatchUnseededFilter'))"),
```

Bit (v12, v13, v14)	Bit (v15, master)	Filter	Collection the filter runs on	Can the bit be set?
12	14	hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter	hltEgammaCandidates	**yes**

13	15	`hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter`	hltEgammaCandidates	**yes**
14	16	`hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter`	hltEgammaCandidatesUnseeded	**no**
15	17	`hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter`	hltEgammaCandidatesUnseeded	**no**
16	18	`hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter`	hltEgammaCandidates (2018 and 2022 menus only)	**yes for 2018 and 2022 menus; no object in 2023 and later menus**
17	19	`hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter`	hltEgammaCandidatesUnseeded (2018 and 2022 menus only)	**no**

All other photon bits (single-photon HE filters, EB tight-ID filters, the Mu17_Photon30 photon leg, the VBF photon filter) were checked in the master menu: every one of them runs on `hltEgammaCandidates`, so they are unaffected. The two bits inserted at positions 9 and 10 in v15 (`hltEG45/50EBTightIDTightIsoTrackIsoFilter`) explain the shift by two between the two numbering columns. As decimal values the dead bits are 16384, 32768 and 131072 in v12 to v14 and 65536, 131072 and 524288 in v15 and master.

4. Why there is never a match: the full chain



TriggerObjectTableProducer: sel is tested first; qualityBits only for objects that passed sel; skipObjectsNotPassingQualityBits drops objects with no bit.

Figure 1. Flow of a photon trigger object from the HLT to `TrigObj_filterBits`. Unseeded-leg objects never reach the bit evaluation.

Every link of the chain was checked in the source code, from the HLT filter to the NanoAOD branch.

1. **The HLT filter tags the object with the unseeded collection and type 92.** `hltDiEG22EtEta2p55UnseededFilter` is an `HLT1Photon` with `inputTag = hltEgammaCandidatesUnseeded`, `triggerType = 92`, `saveTags = True`. Every later filter of the leg is an `HLTEgammaGenericFilter` or `HLTEgammaGenericQuadraticFilter`, both instances of `HLTGenericFilter` (`HLTrigger/Egamma/plugins/HLTGenericFilter.cc`): line 30 reads `l1EGCand`, line 139 calls `filterproduct.addCollectionTag(l1EGTag_)`, line 143 saves the objects with `trigger::TriggerCluster` (value 92, `DataFormats/HLTReco/interface/TriggerTypeDefs.h`). So the saved objects are of type 92 and are referenced through the collection `hltEgammaCandidatesUnseeded`.
2. **PAT copies the collection name into the object.** `PATTriggerProducer` (`PhysicsTools/PatAlgos/plugins/PATTriggerProducer.cc`, line 699) sets `triggerObject.setCollection(handleTriggerEvent->collectionTagEncoded(iC))`, which yields `hltEgammaCandidatesUnseeded::HLT` for these objects, and attaches the filter labels of the filters that accepted them.
3. **MiniAOD keeps the collection name.** `slimmedPatTrigger` (`PhysicsTools/PatAlgos/python/slimming/slimmedPatTrigger_cfi.py`) only packs the filter labels and the four-vector; NanoAOD's `unpackedPatTrigger` restores the labels. Objects from both e/gamma collections reach the table producer with their original collection name.
4. **The collection test is an exact label comparison.** In the string cut, `coll('X')` calls `TriggerObject::hasCollection` (`DataFormats/PatCandidates/interface/TriggerObject.h`, line 158; `src/TriggerObject.cc`, lines 54 to 71, identical in v13, the fork and upstream master). There is no wildcard or prefix logic: the stored collection is split into label, instance and process, and the label must be equal to the requested one.

5. **The selection is applied before the bits.** In `TriggerObjectTableProducer::produce` (`PhysicsTools/NanoAOD/plugins/TriggerObjectTableProducer.cc`, lines 151 to 160; line numbers identical in the fork and in upstream master) the loop first tests `sel.match(obj)`, the `sel` string, and only then evaluates `qualityBits`.
6. **The later duplicate-merging cannot rescue the object.** The producer merges objects of the same id with identical p_T and direction and ORs their bits, but only among objects that already passed the selection.

```
bool TriggerObject::hasCollection(const std::string& collName) const {
  if (collName == collection_) return true;
  const edm::InputTag collectionTag(collection_);
  const edm::InputTag collTag(collName);
  if (collTag.process().empty()) {
    if ((collTag.instance().empty() && collectionTag.instance().empty()) ||
        collTag.instance() == collectionTag.instance()) {
      return (collTag.label() == collectionTag.label());   // "hltEgammaCandidatesUnseeded" != "hltEgammaCandidate
s"
    }
  }
  return false;
}
```

```
for (const auto &obj : trigObjs) {
  for (const auto &sel : sels_) {
    if (sel.match(obj)) {                                    // sel string: coll('hltEgammaCandidates') ...
      selected_bits[sel.id][&obj] = sel.qualityBits(obj);       // bits are computed only here
      if (sel.skipObjectsNotPassingQualityBits ? (selected_bits[sel.id][&obj] != 0) : true)
        selected.emplace_back(&obj, &sel);
    }
  }
}
```

A concrete event: a seeded candidate A passes the leading leg; it carries the labels of filters 1 to 8 and enters the table with bits 12 and/or 13. The same cluster is also reconstructed as unseeded candidate B, which carries the labels of filters 9 to 15 of the sub-leading leg. B is rejected by `coll('hltEgammaCandidates')`. A does not carry any unseeded label. The result is a photon `TrigObj` whose bits 14 and 15 are always zero, which is exactly the observation.

5. How many triggers and bits are affected

The question was checked menu by menu, by searching every path definition for the sequences and filters that carry the unseeded photon bits. The count of dead photon bits is three in every NanoAOD version since the bits exist. The number of HLT paths behind those bits depends on the menu era.

Menu era (source)	Paths whose unseeded leg has a dead NanoAOD photon bit	Dead bits, v12 to v14 numbering (v15 and master)

2018, Run 2 (`CMSSW_10_6_X` GRun V3)	`HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90_v13`
	`HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass95_v13`
	`HLT_Diphoton30_18_R9IdL_AND_HE_AND_IsoCaloId_NoPixelVeto_v2`
	`HLT_Diphoton30_18_R9IdL_AND_HE_AND_IsoCaloId_NoPixelVeto_Mass55_v2`
		14, 15 (16, 17)
		17 (19)

2022 (`CMSSW_12_4_X` GRun V171, `CMSSW_12_6_X` GRun V16)	`HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90_v15`
	`HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass95_v15`
	`HLT_Diphoton30_18_R9IdL_AND_HE_AND_IsoCaloId_v3`
	`HLT_Diphoton30_18_R9IdL_AND_HE_AND_IsoCaloId_Mass55_v3`
		14, 15 (16, 17)
		17 (19)

2023 (`HLT_2023v12`, `CMSSW_13_3_1_patch1` GRun and FULL)	`HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90_v17 / _v19` `HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass95_v17 / _v19`	14, 15 (16, 17)
2024 and 2025 (fork master GRun, 2025 menu)	`HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90_v24` `HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass95_v24`	(16, 17)

In words: the main H to gamma gamma triggers, the Mass90 and Mass95 paths, lose the sub-leading-leg match in every data-taking year and every MC campaign processed with NanoAOD v12 or later, including Run 2 inputs re-processed with those releases. In the 2018 and 2022 menus the `Diphoton30_18` pair of paths is affected as well through bit 17 (19). Mass90 and Mass95 share the same filters, so the fix restores both at once; the same holds for the `Diphoton30_18` pair.

Related gaps that this fix does not address

- **Stale names for the Diphoton30_18 paths from 2023 on.** In the 2023 and later menus these paths were re-tuned and their filters renamed (for example the unseeded last filter is now `hltDiEG18TrackIso60Iso60b40eCaloId11b32eHE10b9eR9Id50b90eTrackIsoUnseededFilter`, and the sequence is `HLTDiphoton3018R9Id50b90eANDHE10b9eANDCaloId11b32eANDIso60b40eSequence`). NanoAOD bits 16 and 17 (18 and 19) still carry the 2022 names, so for 2023 and later data they match nothing on either leg. That needs new bit entries, a separate change.
- **Paths with no photon bits at all.** The low-mass diphoton paths `HLT_Diphoton2{0,2,4}_1{4,6}_eta1p5_R9IdL_AND_HE(T)_AND_IsoTCaloIdT` (six paths, present since 2022), the six `HLT_DiphotonMVA14p25_*_Mass60` paths of the 2025 menu, and `HLT_DoublePhoton33_CaloIdL`, `HLT_DoublePhoton70` and `HLT_DoublePhoton85` have no entry in the photon bit list. Their legs cannot be matched from NanoAOD for a different reason, and adding them is a request to the relevant POG.

6. Explanations that were ruled out

- **Misspelt or renamed filter names.** All four diphoton 30/22 filter names used by NanoAOD are defined, with `saveTags = True`, in every menu examined: `HLT_2023v12_cff.py`, `HLT_GRun_cff.py` and `HLT_FULL_cff.py` of `CMSSW_13_3_1_patch1`, the GRun menus of `CMSSW_10_6_X`, `CMSSW_12_4_X` and `CMSSW_12_6_X`, and `HLT_GRun_cff.py` and `HLT_FULL_cff.py` of master. The names have not changed since the 2018 menu.
- **Objects missing from MiniAOD.** The unseeded objects are stored in `slimmedPatTrigger`: the electron selection has relied on `coll('hltEgammaCandidatesUnseeded')` since June 2023 and its unseeded `CaloIdL_MW` bit is populated in production; the author of PR 41827 observed the increase in trigger-object count that comes from admitting that collection.
- **Wrong object type.** The unseeded filters save their objects as `TriggerCluster` (92), the type the photon selection requires.
- **Era-dependent overrides.** No era modifier touches the Photon selection or its bits in `triggerObjects_cff.py`.
- **v13-specific breakage.** The v13 release and the `CMSSW_13_3_X` tip have byte-identical NanoAOD configuration, and the same selection is in every other production branch (section 7).

7. History and affected NanoAOD versions

- **8 February 2023, PR 40711** ("diphoton_hlt_filters_added"): the six diphoton photon bits were added to `triggerObjects_cff.py` for "the next iteration of the NANOAOD production". The photon selection was left at `coll('hltEgammaCandidates')`. A reviewer asked whether the bits could be OR-ed; the author kept them separate.
- **8 June 2023, PR 41827** ("Add CaloIdL_MW unseeded filter to nanoAOD filterBits"): the same mechanism was recognised for electrons. The author added "the collection `hltEgammaCandidatesUnseeded`, which are the objects on which the selection of the unseeded trigger leg is made" to the Electron selection. The Photon selection was not updated.
- **November and December 2023**: the 13_3_X backports made "for the production of NanoAODv13" (PR 43292, 43286, 43470 and others) changed filter bits and electron IDs but not the photon selection.
- **Today**: upstream master (`ed35654b1`, 4 November 2026) still has `sel = "type(92) && pt > 15 && coll('hltEgammaCandidates')"` at line 90, and `CMSSW_15_0_X` has the same. The producer and the collection matcher are unchanged there. No upstream issue or PR about photon unseeded objects was found.

NanoAOD version	Release	Photon selection	Unseeded diphoton bits present	Dead bit indices (0-based)
v12	CMSSW_13_0_X (13_0_13)	hltEgammaCandidates only	yes	not re-derived here
v13	CMSSW_13_3_1_patch1	hltEgammaCandidates only	yes	14, 15, 17
v14	CMSSW_14_0_X	hltEgammaCandidates only	yes	14, 15, 17
v15	CMSSW_15_0_X	hltEgammaCandidates only	yes	16, 17, 19

master (fork, 2025-05-06)	15_1_X cycle	hltEgammaCandidates only	yes	16, 17, 19
upstream master (2026-11-04)	current	hltEgammaCandidates only	yes	16, 17, 19 (bit list identical to the fork)

Impact. Any analysis that matches the sub-leading photon of the diphoton triggers to its HLT leg through `TrigObj` finds no candidate: trigger matching and leg-by-leg efficiency studies on NanoAOD (for example in HiggsDNA-based H to gamma gamma workflows) can only use the leading-leg bits. Existing v12 to v15 files cannot be repaired after the fact, because the unseeded objects were never written. Only a new NanoAOD pass with the corrected configuration restores the information.

8. The fix and its expected effect

Commit `1699a39f7` (fork branch) and its cherry-pick `aebf3eb64` (on upstream master) change one line of `PhysicsTools/NanoAOD/python/triggerObjects_cff.py`:

```
-            sel = cms.string("type(92) && pt > 15 && coll('hltEgammaCandidates')"),
+            sel = cms.string("type(92) && pt > 15 && (coll('hltEgammaCandidates') || coll('hltEgammaCandidatesUnse
eded'))"),
```

Why this form. `hasCollection` has no wildcard, so the two labels must be OR-ed explicitly. This mirrors the Electron selection, which has used the same construct in production since NanoAOD v12.

What changes in the output. Unseeded objects now enter the photon branch of the producer. Because `skipObjectsNotPassingQualityBits` is true, an unseeded object is written only if it sets at least one photon bit, and the only photon bits whose filters run on the unseeded collection are the diphoton sub-leading-leg bits. So the photon `TrigObj` list grows only in events in which an unseeded candidate passed one of those filters. When the unseeded candidate coincides with a seeded one (same p_T and direction within 10^{-6}), the producer merges them into one entry with the OR of the bits; otherwise a second entry appears, as already happens for electrons. All other photon bits are unaffected.

What does not change. The bit list, the bit numbering, the documentation strings, the electron, muon, tau and jet selections, and the NanoAOD event content are untouched.

9. Validation plan

The fix could not be executed in the environment used for this report, which has the repository but no CMSSW runtime. The following checks are proposed in a CMSSW area, for the pull request or for a local test.

1. Run the standard NanoAOD relval workflows that read 2023 and 2024 MiniAOD, for example `runTheMatrix.py -l 2500.1,2500.11,2500.2,2500.21` (names `NANOmc130X`, `NANOdata130Xrun3`, `NANOmc140X`, `NANOdata140Xrun3` in `Configuration/PyReleaseValidation/python/relval_nano.py`), before and after the change.
2. Count, on the output, photon trigger objects with the unseeded bits set in events that fired the diphoton path. Before the fix the count is zero by construction; after the fix it must be non-zero and of the same order as the count of events firing the path.
3. Compare `nTrigObj`, the NanoAOD DQM trigger-object histograms and the file size. Only the photon multiplicity in diphoton-triggered events is expected to move; PR 41827 showed the equivalent change for electrons.

```
import uproot, awkward as ak, numpy as np
t = uproot.open("nano.root")["Events"]
a = t.arrays(["TrigObj_id", "TrigObj_filterBits", "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90"])
ph    = a.TrigObj_id == 22
fired = a["HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90"]
for bit in (16, 17):          # 14, 15 for NanoAOD v12 to v14 numbering
    hasbit = ak.any((a.TrigObj_filterBits[ph] >> bit) & 1 == 1, axis=1)
    print(f"bit {bit}: {np.sum(hasbit & fired)} of {np.sum(fired)} triggered events have a matching photon TrigOb
j")
```

10. Pull request: what is ready

Item	State

Fix on the fork's working branch	commit `1699a39f7` on `claude/fervent-lamport-3oiuh6` (based on the fork's May 2025 master), local only
PR branch on today's upstream master	branch `nano-photon-trigobj-unseeded` = upstream master `ed35654b1` (2026-11-04) + commit `aebf3eb64`; cherry-pick applied cleanly; checked out in the worktree `scratchpad/prwt`; local only
PR title and description	file `PR_body.md` in the scratchpad directory, ready to paste; follows the cms-sw template headings
Target	`cms-sw/cmssw` master; the same line is still present there (line 90). A backport to `CMSSW_15_0_X` is worth requesting if another NanoAOD production from that branch is planned

To publish, from the main checkout (the branch object is shared with the worktree):

```
git push -u origin nano-photon-trigobj-unseeded
# then open the PR from santanumahata:nano-photon-trigobj-unseeded to cms-sw:master,
# with the title and body from PR_body.md
```

What reviewers will look at. The change touches the xpog area, so it needs the xpog signature. The bot's RelVal comparison will show differences in `nTrigObj` and in the trigger-object DQM histograms for the NanoAOD workflows, which are the expected ones. Referring to PR 40711 (where the bits came from) and PR 41827 (where the same fix was done for electrons) pre-empts the two most likely questions. The two related gaps in section 5 (2023 names of the Diphoton30_18 paths, missing bits for the low-mass and MVA diphoton paths) are best proposed as separate follow-up PRs so that this one stays a one-line bug fix.

11. Appendix: references, line numbers, sources

Fact	Where to look
Photon selection, fork master before the fix (upstream master: line 90)	`PhysicsTools/NanoAOD/python/triggerObjects_cff.py:86`
Unseeded photon bits, fork master	`PhysicsTools/NanoAOD/python/triggerObjects_cff.py:107,108,110`
Electron selection with both collections (upstream master: line 61)	`PhysicsTools/NanoAOD/python/triggerObjects_cff.py:57`
Photon selection and bits, v13	`CMSSW_13_3_1_patch1:PhysicsTools/NanoAOD/python/triggerObjects_cff.py:83,102,103,105`
Exact-label collection match	`DataFormats/PatCandidates/src/TriggerObject.cc:54-71; interface/TriggerObject.h:158`
Selection applied before bits	`PhysicsTools/NanoAOD/plugins/TriggerObjectTableProducer.cc:139,151-160`
Generic e/gamma filter: collection tag and object type	`HLTrigger/Egamma/plugins/HLTGenericFilter.cc:30,139,143; DataFormats/HLTReco/interface/TriggerTypeDefs.h:95`
PAT sets the collection name; MiniAOD slimmer keeps it	`PhysicsTools/PatAlgos/plugins/PATTriggerProducer.cc:699; PhysicsTools/PatAlgos/python/slimming/slimmedPatTrigger_cfi.py`
Path and sequence, v13 menu	`CMSSW_13_3_1_patch1:HLTrigger/Configuration/python/HLT_FULL_cff.py:119468,118801`
Unseeded filter definitions, master menu	`HLTrigger/Configuration/python/HLT_GRun_cff.py:47223-47225,47281-47302,47362-47383; path at 86983`
2018 and 2022 menus (Diphoton30_18 sequences)	`CMSSW_10_6_X and CMSSW_12_4_X / CMSSW_12_6_X: HLTrigger/Configuration/python/HLT_GRun_cff.py`
NanoAOD relval recipes	`Configuration/PyReleaseValidation/python/relval_nano.py`
Fix	`1699a39f7 (claude/fervent-lamport-3oiuh6); aebf3eb64 (nano-photon-trigobj-unseeded on upstream master); not pushed`

Sources consulted. cms-sw/cmssw pull requests 40711 (diphoton photon bits), 41827 (electron unseeded collection), 43292, 43286 and 43470 (13_3_X backports for NanoAOD v13); upstream branches CMSSW_10_6_X, CMSSW_12_4_X, CMSSW_12_6_X, CMSSW_13_0_X, CMSSW_13_3_X, CMSSW_14_0_X, CMSSW_15_0_X and master; NanoAOD v13 dataset names and global tags from public analysis configurations (uhh-cms/cmsdb v13 campaigns, bhbam/MCProduction_run3, maxgalli/SBI-Htt-EFT, choij1589/CustomNanoAOD) and from the 2022 skim notes of the santanumahata/2023_lowmass_analysis repositories. The CERN TWiki, McM and the NanoAOD documentation site were not reachable from the environment used for this report.